STAGE 1 · DISCOVERY REPORT

Handling Sales Day Demand for the Onward Logistics Order System

Elastic, highly available and private order platform on AWS: findings, options and business value

CLIENT
Onward Logistics

PREPARED BY
Patrick Ndaziramiye

DATE
9 November 2026

MODULE
Compute & Delivery on AWS · Week 3

AWS REGION
eu-west-1 (Ireland)

DELIVERY
CloudFormation + Git sync



At a glance

3×
order traffic on sales days, at unpredictable times

2 → 6
servers, scaled automatically on CPU (target 50%)

2 AZs
survives the loss of a server or a whole data centre

1
public entry point; every server is private

≈ $3.75
per quiet day; a sales day adds ≈ $1.60

Executive summary

On sales days, order traffic reaches about three times a normal day, at times nobody can schedule. Today's fixed fleet either sits mostly idle or slows down at the peak. We recommend one public **Application Load Balancer** in front of an **EC2 Auto Scaling group** in **private subnets across two Availability Zones**. A CPU target-tracking policy adds servers during the rush and removes them afterwards.

A **Regional NAT Gateway** gives the servers outbound-only access to updates. Admin access goes through **Systems Manager**, so no admin ports are open. The whole environment is defined in **CloudFormation** and deployed from Git through **Git sync**, so it can be rebuilt from the repository at any time.

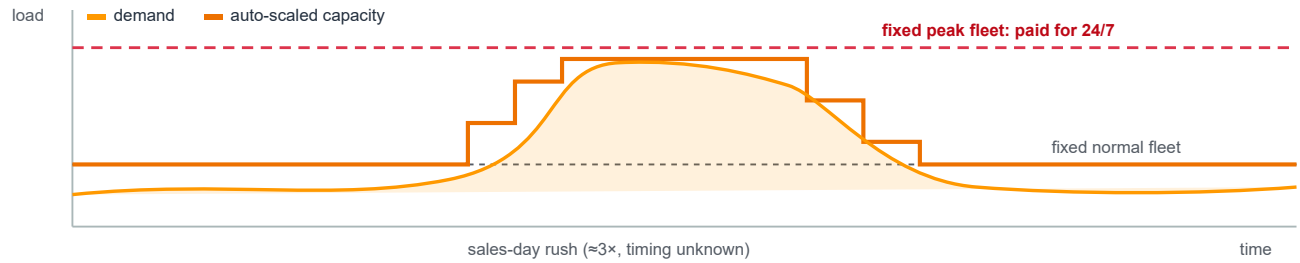
Contents

A1 The problem 2
A2 Common solutions compared 2
A3 Impact of doing nothing 3
A4 Security and resilience patterns 3
A5 Sources 3
Target architecture diagram 4
B1 Chosen approach 5
B2 Services and why 5
B3 Who benefits 6
B4 Cost: quiet vs sales day 7
B5 Trade-offs and roadmap 7
Requirements traceability 6

PART A What we found

1 The problem

Onward Logistics runs its order system on a fixed set of servers. On sales days demand jumps to roughly **three times** normal, and nobody can predict exactly when it will start.



The dilemma today

Size for the peak and pay for idle servers about 95% of the month, or size for a normal day and slow down or fail exactly when merchants need the system most. One server or Availability Zone failure can also take orders offline.

2 Common solutions compared

Approach	How it works	Fit for Onward Logistics
Over-provision for peak	Run peak capacity all the time	Rejected pays for ~3× capacity most of the month
Manual scaling	Operators add servers when they see load	Rejected slow, error-prone, needs someone on call
Scheduled scaling	Add capacity at fixed times	Complement sale timing is unpredictable
Step scaling	Alarms with hand-tuned scaling steps	Not needed more to tune than target tracking
Target-tracking auto scaling	Keep average CPU at a target; scale out *and* in automatically	Chosen hands-off, little tuning, cost follows demand
Serverless (Lambda / Fargate)	The platform scales per request or task	Future brief requires EC2 behind a load balancer
Managed PaaS (Beanstalk)	Platform-managed fleet	Future hides network controls the client wants
CDN / caching in front	Serve static content from the edge	Complement does not help order writes

3 Impact of doing nothing

Lost orders and merchants

Slow pages and errors at the peak turn away customers exactly when demand and revenue are highest.

Damaged trust

Merchants who rely on Onward on their busiest day move to competitors after one bad sales day.

Night-time firefighting

Engineers add servers by hand under pressure, with a high risk of mistakes and burnout.

Paying for idle capacity

The only "safe" alternative is a peak-sized fleet all month, about 3× the compute bill.

Single point of failure

A single server or data-centre (AZ) outage takes the order system offline.

Exposed servers

Publicly reachable servers and open admin ports widen the attack surface.

4 Security and resilience patterns

Public ALB, private compute

Only the load balancer has a public address. Servers sit in private subnets with no public IPs.

Security group chaining

The app port accepts traffic only from the ALB's security group: no CIDR rules, no port 22.

Outbound-only internet

A Regional NAT Gateway lets servers fetch updates but blocks all inbound connections.

Multi-AZ self-healing

Two AZs, minimum 2 servers, ELB health checks: failed servers are replaced automatically.

No admin ports

SSM Session Manager and Run Command over HTTPS replace SSH and bastion hosts.

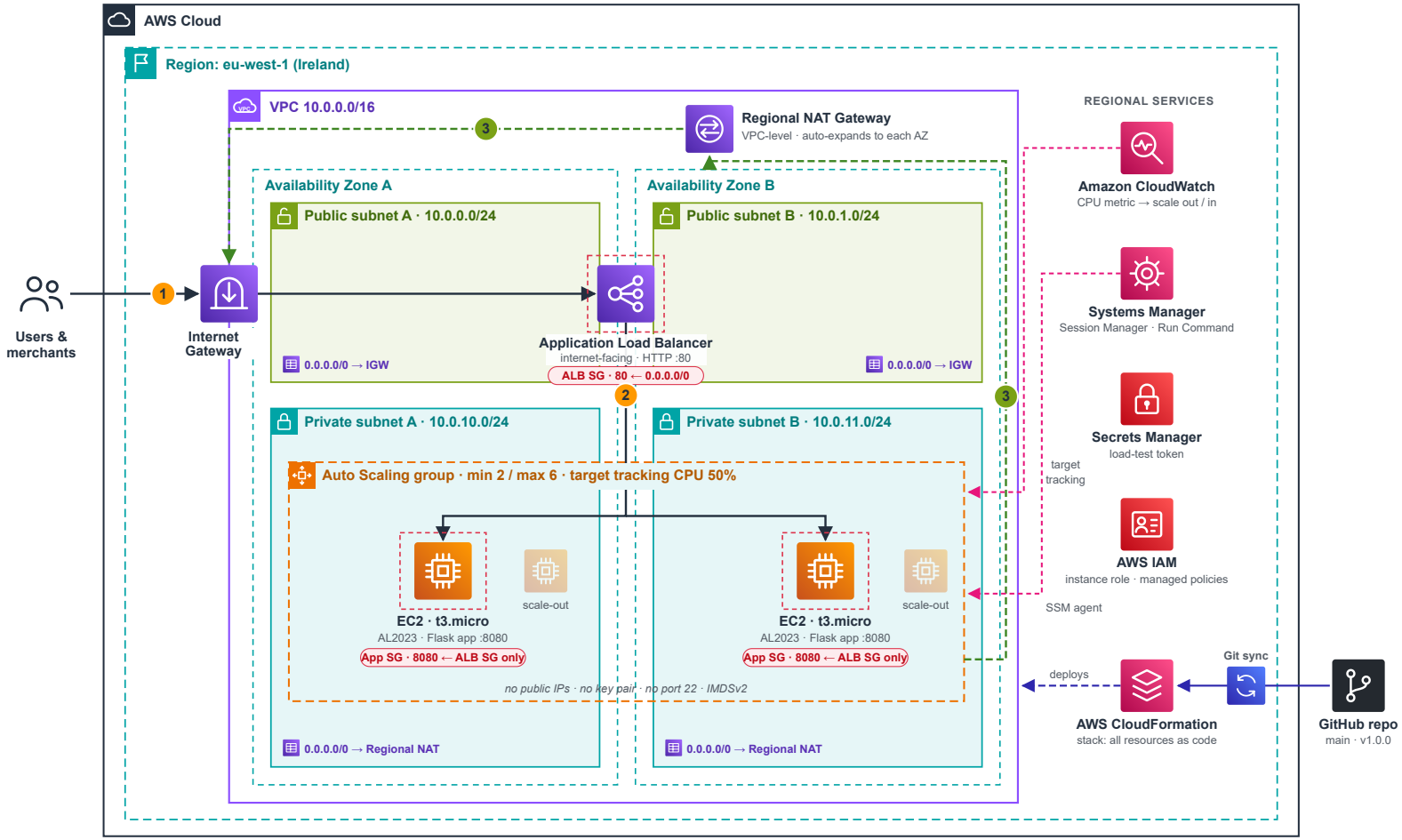
Secrets and least privilege

Token in Secrets Manager, IAM managed policies only, IMDSv2 enforced.

5 Sources

- Target tracking scaling policies for Amazon EC2 Auto Scaling
- Elastic Load Balancing health checks for an Auto Scaling group
- What is an Application Load Balancer?
- Regional NAT gateways for automatic multi-AZ expansion
- AWS CloudFormation Git sync
- AWS Systems Manager Session Manager
- AWS Well-Architected Framework: Reliability, Security and Cost Optimization pillars

TARGET ARCHITECTURE Onward Logistics order system on AWS



Traffic flows

1. Users → Internet Gateway → Application Load Balancer (the only public endpoint)
2. ALB → EC2 on :8080 in both AZs. Health check **GET /health** every 15 s
3. EC2 → Regional NAT → IGW, outbound only (updates, git, pip, SSM)

Groups

- VPC
- Region / Availability Zone
- Public subnet (ALB only)
- Private subnet (servers)

Controls

- Auto Scaling group (min 2 / max 6)
- Security group boundary
- Control plane (scaling, SSM)
- Git → Git sync → CloudFormation

PART B Business value

1 Chosen approach

A single internet-facing **Application Load Balancer** is the only public entry point. Behind it, an **EC2 Auto Scaling group** of small `t3.micro` servers runs in **private subnets across two Availability Zones**. A **target-tracking policy** keeps average CPU near **50%**, adding servers (up to 6) during a sales rush and removing them (down to 2) afterwards. Nobody has to touch it.

Each new server installs the order application itself at boot from a pinned Git release. Each page shows which server and AZ answered. A **Regional NAT Gateway** lets the servers download updates without being reachable from the internet. Everything is defined in one **CloudFormation** template that deploys automatically from Git through **Git sync**, so the environment can be rebuilt from the repository.

Scale out

CPU above 50% → CloudWatch alarm → ASG launches servers (max 6).

Scale in

After ~15 min of low CPU the ASG removes extra servers (min 2).

Self-heal

Failed /health → ALB marks the server unhealthy → ASG replaces it.

2 Services and why

AWS service	Role in the solution	Why this choice
Amazon VPC	Public and private subnets in 2 AZs, route tables	Network isolation and AZ fault tolerance
Application Load Balancer	Single public address, spreads requests, health checks	Stable endpoint while servers come and go
EC2 Auto Scaling	Keeps 2 to 6 healthy servers, target tracking at 50% CPU	Elastic and self-healing, scales both ways
Launch Template + EC2	AL2023, t3.micro, IMDSv2, UserData installs the app	Every server built identically, Free Tier sized
Amazon CloudWatch	CPU metric and alarms behind target tracking	Managed by the policy, nothing to hand-tune
Regional NAT Gateway	Outbound-only internet for all AZs	One resource, expands into each AZ automatically
AWS Systems Manager	Session Manager shell, Run Command load test	Admin access with zero open ports
AWS Secrets Manager	Generated token for the load-test endpoint	Secret never stored in code or Git
AWS IAM	Instance role with managed policies only	Least privilege, auditable, no inline policies
CloudFormation + Git sync	Whole stack as code, deployed on every push	Repeatable, reviewable, rebuildable from Git

3 Who benefits

CTO: elasticity

Capacity follows demand automatically, in both directions, with no one touching it.

Head of Security: isolation

One public entry point, private servers, no admin ports, secrets kept out of code.

Head of Ops: resilience and visibility

Survives server or AZ loss, self-heals, and every page shows which server answered.

Finance Lead: pay for use

2 small servers on quiet days and more only during the rush. MaxSize caps the worst case.

Customers and merchants

Fast, available ordering on the days that matter most.

Engineering team

Everything in Git: reviewable changes, automatic deploys, one-step rebuild.

✓ Requirements traceability

Stakeholder	Requirement	Design answer
CTO	Adjust automatically to CPU spikes, scale back in	ASG + target tracking on `ASGAverageCPUUtilization` = 50%
Head of Security	One public entry point, servers unreachable, no admin ports	Public ALB; EC2 in private subnets; App SG allows ALB SG only; SSM, no SSH
Head of Ops	Survive server / AZ loss, see which server answered	2 AZs, min 2, ELB health checks; page shows instance ID and AZ
Finance Lead	Cost on a quiet vs peak day, and why	Small burstable instances, min 2 / max 6, cost model below
Platform	Private servers can download updates	Regional NAT Gateway, private route 0.0.0.0/0 → NAT
Standards	Everything as code, no inline policies, naming and tags	CloudFormation + Git sync, managed policies, tags on every resource

4 Cost: quiet day vs sales day

Approximate eu-west-1 on-demand list prices; confirm with the AWS Pricing Calculator before sign-off.

Component	Unit price	Quiet day	Sales day
EC2 t3.micro	$0.0114 / instance-hour	48 h → $0.55	72 h → $0.82
T3 unlimited surplus	$0.05 / vCPU-hour	$0.00	≤ $1.00
Application Load Balancer	$0.0252 / h + LCU	$0.65	$0.80
Regional NAT (2 AZs) + data	$0.048 / h per AZ + $0.048 / GB	$2.35	$2.45
EBS, Secrets, CloudWatch	various	$0.20	$0.27
Total per day		**≈ $3.75**	**≈ $5.35**

Compute is the smallest line

The fixed ALB and NAT dominate. A whole sales day adds about $1.60, and a fixed fleet of 6 would cost about 3× the compute all month.

Built-in cost guard rail

`MaxSize = 6` caps the worst case; scale-in returns to 2 servers after the rush.

5 Trade-offs and roadmap

Trade-off accepted now	Why it is acceptable	Mitigation / next step
Orders kept in memory per server	Demo scope, proves routing and scaling	DynamoDB or RDS for shared, durable orders
HTTP only on the ALB	No domain or certificate in the lab account	ACM certificate, HTTPS listener, 80 → 443 redirect
Scale-in takes ~15 minutes	Deliberate: avoids flapping during a rush	Add scheduled scaling for known campaigns
NAT Gateway is the largest fixed cost	Needed for updates with no inbound exposure	VPC endpoints when traffic grows
On-demand instances only	Simple and predictable for the lab	Savings Plan for baseline, Spot for burst

Now
ALB + private multi-AZ ASG, target tracking, Regional NAT, Git sync

Next
HTTPS with ACM, AWS WAF on the ALB, AWS Budget alerts

Later
DynamoDB orders, CloudFront in front, VPC endpoints

Future
Containers (ECS on Fargate), Spot burst capacity, Savings Plans